

qscreen-filing-tool — Launch Tasks (PDF handoff)

The complete pre-fill checklist for a human operator.

Six tasks. In order. Each includes the destination URL, the verbatim copy to paste, and the time estimate. The only non-trivial piece is Task 1 (Show HN first-comment) — everything else is copy/paste from this document or from the matching card on the launch center at <https://qscreen-filing-tool.vercel.app/launch.html>.

Total estimated hands-on time: ~2 hours over 30 days. If you only do one: Task 1.

Task 1 — Post Show HN (15 min total)

Highest leverage. Nothing else moves the star metric until this lands.

Destination

<https://news.ycombinator.com/submit>

Fields to fill

Title (paste verbatim):

qscreen-filing-tool: open-weight PDF → lossless filing JSON for financial reports

URL (paste verbatim):

<https://github.com/Mine-FNL/qstocks-filing-tool>

First comment (paste as your first comment immediately after posting)

We built an open-source Python engine that turns a PDF financial report into a schema-stable, audit-traceable JSON object. ~3 s per filing, \$0 API cost, math-identity gate refuses to ship self-contradictory records.

Three architectural bets:

1. Deterministic-first extraction. PDF tables are read in code (pdfplumber); the model only fills gaps (audit opinion, notes, segments). A 270M local model produces the same numerical contract as GPT-4o.
2. Math-identity gate. If Assets $\neq$ L + E within $\pm 2\%$, the JSON doesn't ship. 11 cross-cutting + 25 issuer-specific pre-flag rules run before save. Blocked records stay on disk with evidence pointing at the offending line item.
3. SHA-256 cross-filing fingerprints + Sigstore keyless signing + SLSA Build L3. Re-ingesting the same PDF on any engine commit produces bit-identical JSON, attestable back to the engine commit.

124-check public bench (Qatar-listed universe, 80.6 % pass rate, regression-gated). 499 tests. Cross-platform CI. Multi-arch container on ghcr.io.

Try it in 30 seconds (no PDF, no API key):

```
pip install https://github.com/Mine-FNL/qstocks-filing-
tool/releases/latest/download/qscreen_filing_tool-1.6.0-py3-none-any.whl
qscreen-demo
```

Repo: <https://github.com/Mine-FNL/qstocks-filing-tool>

Whitepaper: <https://github.com/Mine-FNL/qstocks-filing-tool/blob/main/whitepaper/whitepaper.pdf>

Live bench: <https://mine-fnl.github.io/qstocks-filing-tool/demo.html>

Happy to dig into any of the design choices in the comments.

After posting

- Stay online for **60–90 min** and reply to every comment within 5 min
- Have backup slides for every live-demo segment (the architecture diagram at [campaign/assets/architecture-16x9.jpg](#) is the best one to attach)
- For questions about specific tech choices, link to the relevant whitepaper section: <https://github.com/Mine-FNL/qstocks-filing-tool/blob/main/whitepaper/whitepaper.md>

Task 2 — Email David Tauriello at XBRL US (5 min)

Highest-leverage research/partner lead per [campaign/conference/sponsor-enrichment.md](#) .

Destination

Open `qscreen-filing-tool.vercel.app/launch.html`, scroll to the **XBRL US partner intro** card, click **Open mail**. The mail client opens with the address, subject, and body pre-filled. Edit the salutation before sending.

Recipient: `david.tauriello@xbrl.us` **Subject:** `qscreen-filing-tool – upstream extractor for your AI Connector MCP server (community contribution)`

Body (paste verbatim)

Hi David,

I lead the `qscreen-filing-tool` project – an open-source Python engine that turns a PDF financial report into a clean iXBRL-tagged JSON object, with a math-identity gate that refuses to ship self-contradictory records ($\text{Assets} \neq \text{L} + \text{E}$ within $\pm 2\%$).

When I saw that XBRL US open-sourced the AI Connector / MCP server in 2026 – providing as-filed XBRL data to consumers – I thought there was a natural upstream connection: `qscreen` could be the deterministic ingestion layer that feeds the Connector cleaner iXBRL-tagged records. Today the Connector pulls from EDGAR; the math-identity gate is a property the upstream could enforce.

Three concrete next steps I can offer:

1. A joint demo: ingest a public 10-K through `qscreen`, pipe the JSON into the AI Connector, show the Connector pulling from a cleaner upstream than EDGAR-direct.
2. A joint whitepaper: "Deterministic-first ingestion for as-filed XBRL data" – 6,000-word write-up that documents the architecture and the gate-result distribution on XBRL-US-tagged filings.
3. A small grant for the UAE jurisdiction profile (\$15k sponsor tier covers ~3 weeks of work); XBRL US could co-sponsor in exchange for the profile shipping with XBRL-US-flavored pre-flag rules.

The whole project is MIT-licensed, supply-chain-hardened (SBOM + Sigstore + SLSA L3), and ships a public 124-check regression bench. Happy to set up a 30-min call.

Repo: `https://github.com/Mine-FNL/qstocks-filing-tool`
 Live bench: `https://mine-fnl.github.io/qstocks-filing-tool/demo.html`
 Whitepaper: `https://github.com/Mine-FNL/qstocks-filing-tool/blob/main/whitepaper/whitepaper.pdf`

– [Your name]

After sending

- Within 7 days, follow up via LinkedIn: <https://www.linkedin.com/in/davidtauriello>
- If no reply in 14 days, send a one-line bump referencing the AI Connector MCP server angle (per `sponsor-enrichment.md §12`)

Task 3 — Set dev.to API key (5 min)

Unlocks one of three auto-publish channels. Required once.

Step 1 — Create dev.to account (if needed)

<https://dev.to/>

Step 2 — Generate API key

<https://dev.to/settings/extensions> → “DEV Community API Keys” → “Generate API Key”

Step 3 — Add as repo secret

<https://github.com/Mine-FNL/qstocks-filing-tool/settings/secrets/actions/new>

- **Name:** `DEV_TO_API_KEY`
- **Value:** paste the API key
- **Save**

Step 4 — Test the workflow

From the repo root:

```
gh workflow run publish-dev-to.yml --ref main
```

Step 5 — Verify

Search your dev.to profile for “qscreen-filing-tool” — the article from `campaign/channels/dev-to.md` should be live.

Optional — pin to profile

Once the article lands, pin it via dev.to dashboard → “My Posts” → “Pin to Profile” so it’s at the top of your dev.to home page.

Task 4 — Set Hashnode API key (5 min)

Same flow as Task 3, with Hashnode.

Step 1 — Create Hashnode account (if needed)

<https://hashnode.com/>

Step 2 — Generate token

<https://hashnode.com/settings/developer> → “Personal Access Token” → “Generate”

Step 3 — Find your publication ID

Go to your Hashnode dashboard. The publication ID is the last segment of your publication dashboard URL (it's a UUID).

Example: if your dashboard URL is `https://hashnode.com/dash/your-publication-name/...`, the publication ID is the part after `dash/your-publication-name/...` or in the page source.

Step 4 — Add two repo secrets

<https://github.com/Mine-FNL/qstocks-filing-tool/settings/secrets/actions/new> (twice)

- Name: `HASHNODE_TOKEN` Value: paste the access token
- Name: `HASHNODE_PUBLICATION_ID` Value: paste the UUID

Step 5 — Test the workflow

```
gh workflow run publish-hashnode.yml --ref main
```

Step 6 — Verify

The article from `campaign/channels/article-medium-specs.md` should be live on your Hashnode publication.

Task 5 — Submit PyCon US 2027 talk (30 min)

Deadline: ~late-October 2026 per `cfp-enrichment.md`. Miss this and you wait another year.

Destination (URL goes live ~mid-August 2026)

<https://us.pycon.org/2027/speaking/>

Abstract (paste verbatim into the CFP form)

Numbers never pass through the model: deterministic-first PDF extraction

Every "PDF to JSON" pipeline I've seen treats the language model as the source of truth for numbers. That's a category error. Numbers live in tables; the model hallucinates 5–10 % of them.

This talk is the postmortem of six months of building an extraction pipeline for financial filings where the production requirement was: "re-ingest the same PDF on any engine commit and get byte-identical JSON, cryptographically attestable back to the engine commit that produced it."

Three architectural bets made it work:

1. Deterministic-first extraction. PDF tables are read in code (pdfplumber); the language model only fills gaps (audit opinion, notes, segment labels). Numbers never pass through the model – so a 270M local model produces the same lossless contract as a frontier cloud model.
2. A math-identity gate that runs before `save()`. If $\text{Assets} \neq \text{L} + \text{Equity}$ within a sector-aware tolerance band, the record doesn't ship – and the evidence points at the offending line item. Eleven cross-cutting + twenty-five issuer-specific pre-flag rules run before every write.
3. SHA-256 cross-filing fingerprints + Sigstore keyless signing + SLSA Build L3 attestation. Re-ingesting the same PDF on any engine commit produces bit-identical JSON, cryptographically attestable back to the engine commit.

The bench is public: 124 cross-filing checks against the current Qatar-listed universe, regression floor 100/124 (80.6 %). Every PR that drops the floor fails CI.

We'll show the architecture, the failure modes we hit on the way (non-deterministic JSON, silently-shipped internal contradictions, opaque provenance), and the three properties – deterministic extraction, math-identity gate, fingerprintable provenance – that solved them. We'll also cover what doesn't work: image-only PDFs, heavy footnote-as-data filings, and the audit-opinion classification ceiling.

Speaker bio (≤ 200 chars)

The qscreen-filing-tool maintainers build open-source data-extraction pipelines for the QSE-listed universe (55 tickers, IFRS / IFRS-for-SME). 499 tests, 80.6 % bench, supply-chain hardened (SBOM + Sigstore + SLSA L3).

Pitch line (≤ 200 chars, fills the optional “elevator pitch” field)

A deterministic-first extraction pipeline where numbers never pass through the model. 270M local model = same contract as GPT-4o. Math-identity gate blocks self-contradictory records. SHA-256 + Sigstore + SLSA L3. Public 124-check bench.

Notes

- The CFP form fields may be labelled differently year-to-year. Common labels: “Title”, “Abstract”, “Description”, “Elevator Pitch”, “Speaker Bio”
- If asked for a co-speaker, leave blank — single-speaker is fine for a community talk
- After submitting, save the confirmation email. CFP responses typically arrive in December

Task 6 — Submit NLnet grant application (45 min)

Hard deadline: November 3, 2026.

Destination

<https://nlnet.nl/propose/>

Form fields

Project name: qscreen-filing-tool – UAE jurisdiction profile (NLnet Privacy & Infrastructure line)

Project description (paste verbatim):

This 6-month project builds out the UAE jurisdiction profile of qscreen-filing-tool to bench-grade (top 30 ADX + DFM listed equities, ~7-10 working days of engineering, fully open-source MIT output).

Background: qscreen-filing-tool is an open-source Python engine that turns any exchange's annual or interim report into a schema-stable, audit-traceable JSON object. Six months in production. 499 tests. 124-check public bench at 80.6 %. Hardened supply chain (SBOM + Sigstore keyless signing + SLSA Build L3 attestation, all public).

The default profile ships 55 Qatar-listed tickers (QSE) with 11 cross-cutting + 25 issuer-specific pre-flag rules. The UAE jurisdiction profile is currently a stub – the engineering is bounded and the architecture is jurisdiction-agnostic, so the work is largely data-tables and pre-flag rules.

Why this fits NLnet's mandate: financial-data infrastructure is increasingly important for transparency / privacy / consumer-protection use cases. The open-source release of jurisdiction profiles makes the audit trail inspectable by anyone, not just by the operator.

Deliverables:

- profiles/uae/ populated to bench-grade (~30 tickers)
- 11 + 25 cross-cutting + issuer-specific pre-flag rules
- New golden-set cases for the bench
- Documented architecture decision records (ADRs) for the UAE-specific fiscal calendar + auditor history
- All release artifacts public, signed, and attested

Repo: <https://github.com/Mine-FNL/qstocks-filing-tool>

Whitepaper: <https://github.com/Mine-FNL/qstocks-filing-tool/blob/main/whitepaper/whitepaper.pdf>

Requested budget: €15,000 (covers the UAE jurisdiction profile work; matches the Bronze-tier sponsor commitment at <https://github.com/sponsors/OxBingBong69>)

Milestones (paste verbatim):

M1 (week 1-2): Profile scaffold – JURISDICTION_NAME="UAE", account-code map skeleton, fiscal-calendar overrides for the ADX/DFM interim-period boundaries. Verify qscreen-ingest --jurisdiction uae runs end-to-end on a synthetic filing.

M2 (week 3-4): Tick data – 30 ADX + DFM listed equities with sub-sector, framework (IFRS / IFRS-for-SME / AA0IFI), fiscal year-end, currency (AED).

M3 (week 5-6): Pre-flag catalog – mirror profiles/qatar/pre_flags.py with UAE-specific rules (Dubai Islamic Bank audit-firm history, Emirates NBD sector taxonomy, Emaar Properties intangibles ceiling, etc.).

M4 (week 7-8): Golden-set cases – 8 UAE filing fixtures with hand-verified expected outputs, validating the bench regression floor at 80.6 %.

M5 (week 9-10): Documentation + supply chain – README refresh, ADR records for the UAE-specific design choices, SBOM + Sigstore signing on the new release.

Final deliverable: v1.7.0 of qscreen-filing-tool ships with both Qatar and UAE profiles at bench-grade.

Team bio:

Single maintainer (0xBingBong69) with 6 months of full-time focus on the project; v1.0 through v1.6.0 shipped with zero failed release gates. Background in quant data infrastructure and MENA financial markets.

Advisors (informal): the qscreen-filing-tool maintainer network includes finance-tech practitioners who have agreed to review the UAE pre-flag rules for cultural / regulatory accuracy. Identities on request.

After submitting

- Save the confirmation email
- NLnet typically responds within 8-12 weeks
- If accepted: contract-signing + first milestone payment within 30 days

After all 6 tasks: 30-day launch cadence

The remaining channels have pre-filled copy at qscreen-filing-tool.vercel.app/launch.html. Bookmark the page; each card has **Copy** + **Open** buttons.

Day	Channel	Time	Notes
+1	Lobsters	5 min	Need invitation; cross-post of the HN post
+2	r/MachineLearning	10 min	Self-post; community-friendly tone
+4	r/quant	10 min	Self-post; quant-flavored copy
+5–10	Product Hunt	30 min	Maker launch at 12:01 AM PT; stay online 4 hr
+7	X thread #1	15 min	The launch thread (15 tweets)
+7	dev.to	0 min	Auto-publishes if <code>DEV_TO_API_KEY</code> is set
+7	Hashnode	0 min	Auto-publishes if secrets are set
+8	LinkedIn	10 min	Long-post format
+14	X thread #2	15 min	Architecture deep-dive (10 tweets)
+21	X thread #3	15 min	Bench story (10 tweets)
+24	Hacker Noon	0 min	Cross-post; auto via Dev.to or manual
+28	X thread #4	15 min	Supply-chain hardening (10 tweets)
+30	X thread #5	15 min	Pluggable profiles (8 tweets)
any	Mastodon / Bluesky	5 min each	Federated, dev-friendly, optional

Total estimated hands-on time for the full 30-day cadence: ~2.5 hours over 30 days.

One-page quick-reference

Task	Destination	Pre-fill source	Time	Deadline
1. Show HN	news.ycombinator.com/submit	this PDF, page 2	15 min	ASAP
2. XBRL US email	launch.html → XBRL card	mailto: link	5 min	this week
3. dev.to API key	dev.to/settings/extensions	—	5 min	this week
4. Hashnode API key	hashnode.com/settings/developer	—	5 min	this week
5. PyCon US 2027	us.pycon.org/2027/speaking/	this PDF, page 7	30 min	~late Oct 2026
6. NLnet grant	nlnet.nl/propose/	this PDF, page 8	45 min	Nov 3, 2026

If you only do one: Task 1 (Show HN). 15 minutes. No excuse.

— qscreen-filing-tool maintainers · 0xBingBong69 · MIT-licensed · 2026-09-19